Selecting Reliable LLM Reasoning: Vote-Aware Listwise Ranking over Answer Clusters

Gao Zhe 3036707422

1. Introduction

A 2,085-parameter NumPy model that scores each candidate and pools the evidence over answer clusters selects a correct answer on 83.51% of the test bank (390/467), using 0.81 s of CPU time and 10 KB of assets. Majority voting reaches 69.0% on the matching validation subset; our model reaches 85.4%.

The task gives four Qwen3-0.6B solutions per GSM8K question, their text and a fixed 1024-d embedding of each, and asks for one candidate index. The scorer checks only the final number.

Three findings organise the report:

1. The embedding and the vote structure carry almost all the usable signal. An L2 logistic probe on the embedding adds about 6 points over the best handcrafted model (79.4 to 85.4 on val-hard, 76.9 to 83.0 in CV); a from-scratch n-gram text model adds nothing on top of it.
2. Judging candidates against each other helps, but only a little. Group-centred embeddings and "cross" features (whether a candidate's answer appears as an intermediate value in another candidate's working) give a consistent +1.0 point in five-seed cross-validation, which a single 355-question validation run cannot resolve.
3. Extra computation buys nothing measurable. An MLP probe, four-probe ensembles, boosted trees and a 262k-parameter text model all land within 0.4 points of the final model, far inside the test bank's 1.7-point standard error.

Section 2 fixes the evaluation protocol, Section 3 the method and Section 4 the main results. Sections 5 to 9 answer the five research questions in the brief; Section 10 analyses the remaining errors.

2. Data and evaluation protocol

All development used the training and validation splits only; the test bank was scored once, after the method was frozen (Section 4).

Hard subsets. Training (4,755 questions) and validation (798) are dominated by unanimous groups, where any choice is right. The brief and the data manifest state that every test group has at least two distinct parsed answers, so we apply the same filter, which needs no reference answers, to obtain train-hard (2,114 groups) and val-hard (355 groups). The vote patterns (sorted cluster sizes) are distributed alike in the two subsets (Table 1).

Table 1. Distribution of vote patterns (sorted cluster sizes) in the two hard subsets.

Vote pattern	(3,1)	(2,1,1)	(2,2)	(2,1)	(1,1,1,1)	(1,1,1)	(1,1)
train-hard share (%)	41.9	23.5	10.8	8.9	7.9	5.3	1.8
val-hard share (%)	42.5	24.8	9.6	7.9	5.6	5.6	3.9

Patterns with fewer than four entries contain unparsable candidates (871 in training, 157 in validation).

We report five-fold cross-validation on train-hard (CV, n = 2,114, standard error about 0.8 points) as the primary estimate and val-hard accuracy (n = 355, standard error about 1.9) as the secondary one. Every comparison that informs a decision is repeated over five seeds, which change the fold split and any random initialisation. Single-run val-hard numbers come with a bootstrap 95% interval; paired comparisons use an exact McNemar test.

The final model scores 85.4 on val-hard but 83.0 in CV. Majority voting shows the same gap (69.0 against 66.8), so it is a property of the samples rather than of the model. Reweighting CV accuracy to the val-hard pattern mix leaves it at 82.9, and training each CV fold on 80% of its data costs only 0.3 points, so neither pattern mix nor training-set size explains it. The remaining gap is about 1.2 combined standard errors, consistent with chance.

About 26.5% of candidates use `####` as a Markdown heading. The official `parse_answer` treats `####` as GSM8K's answer marker and reads the first number after the last one, ignoring `\boxed{}`. This changes the parsed answer of 32 training and 8 validation candidates, touching 22 train-hard and 7 val-hard groups; in half of those cases the boxed answer was in fact correct, and 6 train-hard and 2 val-hard groups count as hard only because of it. The effect is small, but it puts a little noise into both the hard filter and the score (an example appears in Section 10).

3. Method

The method scores every candidate, pools the scores of candidates that share an answer, and returns a member of the best-scoring answer. Two linear probes read the embedding, 30 handcrafted features read the text, and a linear listwise ranker combines them (Figure 1).

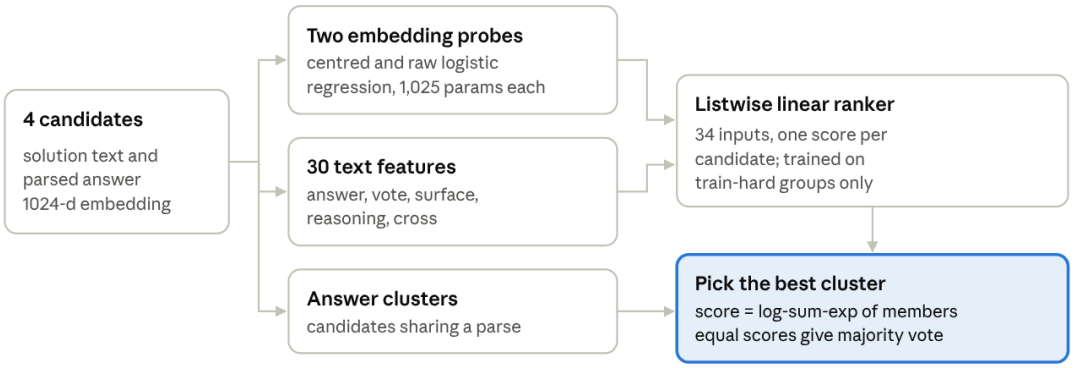


Figure 1. Overview of the selection pipeline.

3.1 Answer clusters and the selection rule. Candidates with the same parsed answer form a cluster. A cluster's score is the log-sum-exp of its members' scores, and the cluster with the highest score is chosen. When all candidate scores are equal, the cluster score reduces to the log of the vote count, so the rule contains majority voting as a special case; learned scores can then outvote the majority only when the evidence is strong.

3.2 Embedding probes. Two L2-regularised logistic regressions ($\lambda = 10$) predict whether a candidate is correct. One sees the raw 1024-d embedding; the other sees the embedding minus the mean of the parsable candidates in its group ("centred"), which removes what the four solutions share and keeps what distinguishes this one. Both are trained on all 18,149 parsable training candidates. The ranker receives each probe's logit and the logit minus its group mean. On training data those logits are produced out-of-fold (5 folds), so the ranker never sees a probe score fitted on the same question.

3.3 Handcrafted features (30, in five groups).

- **answer** (6): the answer parses, is an integer, is non-negative, its magnitude, it equals a number in the question, it exceeds every number in the question.
- **vote** (5): votes for the answer, vote share, number of clusters, belongs to the largest cluster, singleton.
- **surface** (6): log length, length z-score within the group, line count, display-maths blocks, headings, bold spans.

- **reasoning** (9): equations re-computed and how many are wrong, error rate, share of question numbers used, the answer appears as a computed result, "approx/round" words, hedging words ("wait", "assume"), "impossible" words, number of boxed answers.
- **cross** (4): the share of other-answer candidates whose working contains my answer (anywhere, and as an "= result"), and the share of other answers that appear in my working (both ways).

3.4 Listwise ranker. A linear score over the 34 inputs (30 features plus two values per probe) is trained to maximise the probability of the correct cluster:

$$\mathcal{L} = -\log \frac{\sum_{i \in \mathcal{C}^+} e^{s_i}}{\sum_{i \in \mathcal{C}} e^{s_i}}$$

Here $\mathcal{C}$ is the set of parsable candidates in a group and $\mathcal{C}^+$ its correct members, so the loss is exactly the cluster rule of 3.1. Only train-hard groups carry a gradient, and only they are used ($\lambda = 1$, 400 full-batch Adam steps).

3.5 Implementation. Everything is NumPy on one CPU, seed 7506. Training takes 2.7 s; inference loads 10,316 bytes of weights and configuration. Retraining reproduces `weights.npz` bit for bit (identical SHA-256).

4. Main results

The final model beats majority voting by 16.4 points on val-hard and 16.2 points in CV, and scored 83.51% on its single test run. Table 2 builds it up from the baselines.

Table 2. Accuracy from the baselines to the final model

Method	Learned parameters	val-hard (%) [95% CI]	CV on train-hard (%)
First candidate	0	54.9 [49.9, 60.0]	—
Shortest / longest candidate	0	40.8 / 60.6	—
Majority vote	0	69.0 [63.9, 73.8]	66.8 (no training)
Majority vote, non-integer and negative answers dropped	0	73.5 [68.5, 78.0]	—
All handcrafted features, no embedding	31	79.4 [75.2, 83.4]	76.9
One centred probe, votes only	1,028	80.6 [76.3, 84.8]	80.7
One centred probe + features without *cross*	1,054	84.2 [80.6, 87.9] 84.5 ± 0.3	81.3 81.9 ± 0.1
Final: two probes + all features	2,085	85.4 [81.7, 89.0] 85.2 ± 0.3	83.0 82.9 ± 0.2

Exact McNemar tests on val-hard (final right only / other right only) confirm every large step: against majority voting 75/17 ($p = 7\times10^{-10}$), against the filtered vote 61/19 ($p = 3\times10^{-6}$), against handcrafted features 38/17 ($p = 0.006$), against the probe with votes 22/5 ($p = 0.002$). The last two steps are not significant on val-hard alone: against the single probe 10/6 ($p = 0.45$), against two probes without *cross* 7/4 ($p = 0.55$). Their support is the five-seed CV, where they add 1.0 point with a seed spread of 0.1 to 0.2.

We froze the configuration, retrained it (identical weight hash), recorded source and asset hashes, and ran the test bank once: 390 of 467 correct, 83.51%. Before that run, a precision-weighted average of CV and val-hard predicted about 83% with a 95% range of roughly 80 to 87; the outcome agrees. One fresh prediction process took 0.683 s and the scoring process 0.124 s (0.807 s in total) on an AMD Ryzen 7 9800X3D with four threads, against a 30 s limit.

5. Contribution of the reasoning text

Reading the solution lifts accuracy from 72.7 to 85.4 on val-hard (69.9 to 83.0 in CV) over reading only the question and final answer (Table 3), and most of that gain arrives through the embedding. We expected the handcrafted reasoning checks to matter more than they do.

Table 3. Information-source ablation (single run, seed 7506).

Inputs to the ranker	val-hard (%)	CV (%)
Votes + answer features (question and final answer only)	72.7	69.9
… + *surface*	75.5	72.3
… + *reasoning*	74.4	73.6
All handcrafted features	79.4	76.9
One embedding probe + votes	80.6	80.7
Final model	85.4	83.0

Reading only the question and final answer (votes plus answer properties such as "is an integer") reaches 72.7 / 69.9. Every source that reads the solution adds to that, and the embedding probe alone, with nothing but votes beside it, beats all 30 handcrafted features together by 3.8 points in CV. The probe's own ranking is strong before any pooling: taking the argmax of its logit gives 80.0 on val-hard against 80.6 with cluster pooling.

We hashed word, number and symbol 1-3-grams of each candidate into 2^{18} buckets and fitted an L2 logistic regression on the training texts (no vocabulary, no pretrained parts). Alone it ranks candidates within a group at 70 to 72% on val-hard, close to majority voting. Without the embedding it is useful: handcrafted features plus the text model reach 80.5 ± 0.3 / 78.2 ± 0.1, up from 79.4 / 76.9. On top of the final model it adds nothing (85.4 ± 0.2 / 82.9 ± 0.2) while costing 1.01 MiB of weights and about 20 times the training time (61 s against 2.7 s). Whatever the wording carries, the embedding already holds.

6. Length and surface cues

Length misleads simple scorers but not the final model: on pairs of a correct and a wrong candidate of nearly equal length it still ranks the correct one first 86.7% of the time. Our hypothesis was that the probe would partly be a length detector; the evidence says it is not.

Length is a real cue. Picking the longest candidate gives 60.6 on val-hard and the shortest 40.8, and adding the six *surface* features to votes and answer properties gains 2.8 points (72.7 to 75.5). A model could therefore look good by preferring long, well-formatted solutions.

Length-matched pairs separate the explanations. We took every (correct, wrong) candidate pair from the same val-hard group, 1,077 pairs, and asked how often each scorer ranks the correct one higher, split by the difference in log length (Table 4).

Table 4. Pairwise accuracy (%) on (correct, wrong) candidate pairs from the same val-hard group, by difference in log length.

Pairs	n	Final model	Handcrafted only	Probe only	Longest wins
All	1,077	86.7	81.2	81.3	61.3
Length-matched (Δ log length < 0.1)	301	86.7	77.1	79.7	52.8
Correct one is longer	501	89.4	89.6	85.4	100
Correct one is shorter	275	81.8	70.2	75.6	0

The final model loses nothing when length carries no information and only 5 points when length points the wrong way. The handcrafted model drops 4 and 11 points in the same two rows (81.2 to 77.1 and 70.2): it is the one that leans on length.

Ranker weights. Multiplying each weight by its feature's standard deviation on train-hard puts the inputs on one scale. The largest are *integer* (1.22), *non-negative* (0.48), *others' answers appear in my working* (0.45) and the four probe inputs (0.30 to 0.36); log length comes tenth at 0.25. The probe's logit is nearly uncorrelated with length ($r = 0.07$ on validation candidates).

Formatting and wording. Removing headings, bold marks and LaTeX layout tokens from the n-gram model leaves it unchanged (85.2 ± 0.1 against 85.4 ± 0.2 in the full model), so formatting is not what any of our text signals pick up. The single strongest cue is a domain prior learned from training data: in every train-hard and val-hard group, no non-integer or negative answer was ever correct, so such answers are almost never chosen.

7. Comparative judgement across candidates

Comparing candidates within a group helps, but less than expected once the embedding is included: the comparative parts of the final model (a raw probe beside the centred one, plus the cross features) give a consistent +1.0 point in CV, yet the strongest comparison signal we found is worth +2.8 points only to a model without the embedding.

Centring the embedding. Subtracting the group mean before the probe is the cheapest form of comparison. As a single probe, centred input gives 84.2 / 81.3 (val-hard / CV), raw input 83.1 / 81.1, and the two concatenated 83.4 / 81.0. Keeping the two as separate probes is better than either: 84.7 ± 0.1 / 82.6 ± 0.2 over five seeds, without *cross*.

Answers that appear as intermediate values. Inspecting errors suggested that a wrong answer often appears inside another candidate's working, where that candidate kept going. The *cross* features test this directly, and the pattern holds in both splits (Table 5).

Table 5. Probability that a candidate is correct, with and without each cross feature (val-hard; train-hard in parentheses).

Cross feature	P(correct) if present, val-hard (train-hard)	P(correct) if absent
My answer appears in another answer's working	0.35 (0.36), n = 347	0.61 (0.58), n = 997
… as an "= result" there	0.25 (0.27), n = 231	0.60 (0.58), n = 1,113
Another answer appears in my working	0.66 (0.61), n = 415	0.49 (0.50), n = 929
… as an "= result" in my working	0.75 (0.68), n = 271	0.49 (0.50), n = 1,073

An answer that another solution passes through on its way to something else has probably stopped one step early; a solution that passes through another's answer has probably taken the step the question asks for.

Size of the comparative gain. Without the embedding, *cross* lifts the handcrafted model from 76.6 / 74.6 to 79.4 / 76.9. With one probe it is a wash (84.5 ± 0.3 / 81.9 ± 0.1 to 84.0 ± 0.4 / 82.3 ± 0.1); with two probes it adds 0.5 / 0.3 (to 85.2 ± 0.3 / 82.9 ± 0.2). The centred embedding evidently already encodes much of the "stopped early" signal.

Comparisons that did not help. Cosine similarities between a candidate and its same-answer and other-answer neighbours left accuracy unchanged (84.2 / 81.6). A probe trained with the listwise loss directly on centred embeddings, which can learn only from the 2,114 hard groups, was weaker alone (81.1 / 76.7) and added nothing beside the logistic probe (84.5 / 81.3).

8. Calibration and uncertainty

The probability the model gives its chosen cluster tracks its accuracy closely: in the least confident fifth of val-hard it is right 59% of the time, in the most confident fifth always (Figure 2). Errors carry a mean confidence of 0.69, correct choices 0.88.

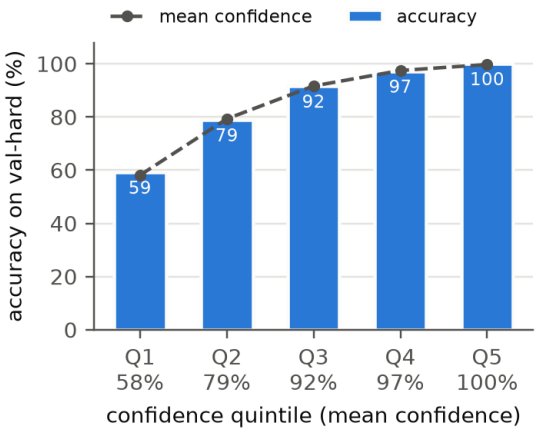


Figure 2. Calibration on val-hard: accuracy against mean confidence in five equal-size confidence bins (71 groups each).

Sources of the gain. Figure 3 splits val-hard by vote pattern. The model gains most where voting has least to say: ties between two pairs (2,2) rise from 41% to 85%, all-different groups (1,1,1,1) from 25% to 70%. In 3-1 splits it overrules the majority only when the evidence is strong, and still gains 7 points.

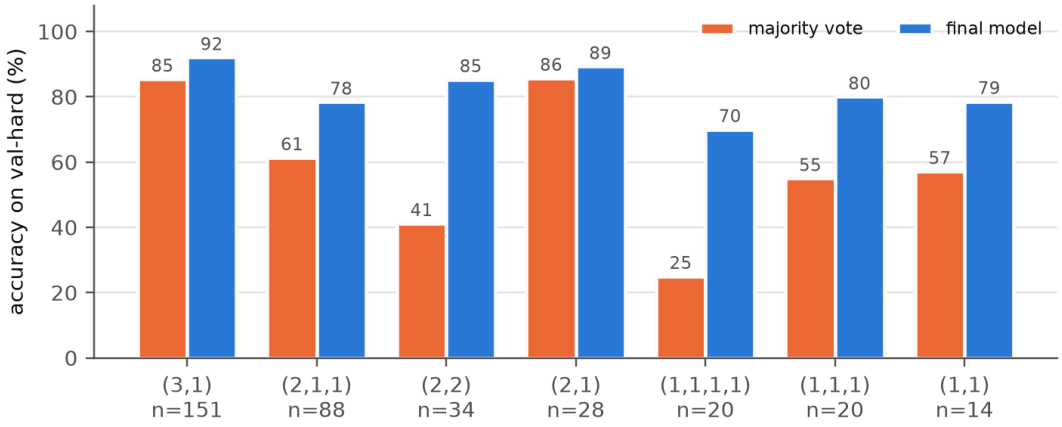


Figure 3. Val-hard accuracy by vote pattern (sorted cluster sizes) for majority voting and the final model; n is the number of groups per pattern.

Disagreement between methods. The final model and the handcrafted-only model pick the same candidate in only 54.4% of val-hard groups. Where they agree the final model is right 88.6% of the time; where they differ it is right 81.5% and the handcrafted model 68.5%. Against the probe-only model they agree in 80.0% of groups, and where they differ the final model wins 85.9% to 62.0%. Disagreement therefore marks harder questions, and the combined model usually sides with the right one. An oracle that took whichever of the final and handcrafted models was right would reach 90.1, so the two are partly complementary, but no input we tried predicts which one to trust (Section 9).

9. Accuracy versus computational cost

Additional computation does not pay off in this setting: every richer variant we tried lands within 0.4 CV points of the final model, while multiplying its parameters by up to 127 (Figure 4 and Table 6). We expected at least the MLP probe or the trees to capture interactions the linear ranker misses.

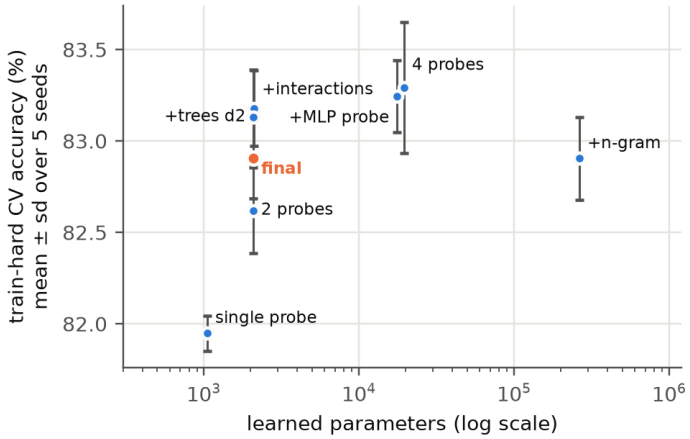


Figure 4. Train-hard CV accuracy (five-seed mean ± sd) against the number of learned parameters (log scale).

Table 6. Richer variants compared over five seeds.

Variant (five seeds)	Learned parameters	val-hard (%)	Extra cost
Final model	2,085	85.2 ± 0.3	training 2.7 s, 10 KB
+ vote-pattern interactions	2,109	85.6 ± 0.5	none
+ boosted trees, depth 2 / 3	2,085 + 100 trees	85.6 ± 0.3 / 85.2 ± 0.1	100 trees
+ MLP probe (16 hidden units)	17,477	84.8 ± 0.4	8× parameters
Four probes (linear, raw, MLP, listwise)	19,531	85.0 ± 0.3	9× parameters
+ n-gram text probe	264,232	85.4 ± 0.2	1.01 MiB, 61 s training, +0.4 s inference

Variants examined. The MLP probe first overfitted (78 to 81 on val-hard); with stronger L2 and a smaller step it beat the linear probe on one seed (84.8 against 84.2) and lost that lead over five. Vote-pattern interactions let the ranker trust votes and probes differently in a 2-2 tie than in a 3-1 split. Gradient-boosted trees (Newton steps on the listwise gradient, starting from the linear scores) can model any interaction among the 34 inputs.

Interpretation. The best variants reach 83.1 to 83.3 in CV against 82.9 ± 0.2 for the final model: one to two seed standard deviations, and a fifth of the test bank's 1.7-point standard error. At that size no single test run can tell them apart. We therefore froze the simplest model inside the top group. Extra capacity found no structure the linear ranker had missed; it made the same mistakes at higher cost.

10. Failure analysis

The model's 52 val-hard errors are near misses on lonely answers: the correct cluster ranks second in 46 and third in 6, and in 40 of the 52 it was proposed by a single candidate.

Error structure (size of the correct cluster, size of the chosen one): (1,1) 17, (1,2) 14, (1,3) 9, (2,2) 5, (2,1) 4, (3,1) 3. Of the 114 val-hard groups whose correct answer has one vote, the model solves 64.9%; this is the bottleneck. The three (3,1) errors are the opposite failure: a three-vote correct answer overruled by a confident singleton.

Four validation examples (chosen automatically as the first error of each pattern; full texts in `runs/diagnostics/failure_examples.md`):

1. **Ambiguous reference (3,1).** "Marion has 1/4 more than what Yun currently has, plus 7", with Yun at 8. Three candidates read "1/4 more than 8" as 8 + 2, giving 17; the reference reads it as 2 + 7 = 9. The model picks 17 with probability 0.89. No reading of the reasoning can recover this one.
2. **Misread question (2,1,1).** The question asks how many students cannot be paired: 48 girls against 46 boys leaves 2. Two candidates subtract the 46 pairable students from all 94, giving 48. The model picks 48 (0.71); the correct answer is second (0.27).
3. **No consensus (1,1,1,1).** Four different answers to a frog-pond question; the model picks 3 (0.56) and ranks the correct 7 third (0.13). The wrong solution invents a step ("frogs that will become tadpoles") that reads as fluently as the right one.
4. **Scorer artefact (2,2).** All four candidates box the answer 1, but two use `####` headings, so the official parser reads them as 1152 (Section 2). The model chooses the misparsed pair. The reasoning was right; the score is not.

11. Reproducibility, resources and compliance

The submitted checkpoint reproduces the reported score without retraining, and retraining reproduces the checkpoint bit for bit (Table 7).

Table 7. Reproducibility and resource summary.

Item	Value
Environment	Python 3.12.14, numpy 2.5.3 (only inference dependency); matplotlib 3.11.2 for figures only
Training	`python train.py`, CPU only, seed 7506, 2.7 s
Inference assets	`config.json` + `weights.npz`, 10,316 bytes; SHA-256 in `runs/freeze_record.json`
Evaluation	official `predict.py` and `score_predictions.py` with OMP/OPENBLAS/MKL threads = 4
Test timing	0.807 s for one fresh prediction process plus scoring (limit 30 s), AMD Ryzen 7 9800X3D
Report numbers	`python experiments.py --seeds` (about 12 min), `python diagnostics.py`, `python make_figures.py`
Code and checkpoint	https://github.com/zjsyxzjsyx/DASE7506-MP2

Compliance. Models were fitted on training inputs and references only; validation served development and selection; the test bank was scored once, after freezing, and no decision depends on it. No external data, pretrained weights, generation or question IDs are used, and the method accepts any bank size. The official `common.py`, `predict.py` and `score_predictions.py` match the hashes in `PACKAGE_MANIFEST.json`.

12. Conclusion

A small linear model over two embedding probes, vote structure and a few comparative text features selects the right answer on 83.5% of the test bank; on the development subsets it beats majority voting by about 16 points (85.4 against 69.0 on val-hard, 83.0 against 66.8 on train-hard), at negligible cost.

Three conclusions follow from the experiments. The fixed embedding, read by a linear probe, carries most of the information in the reasoning, and a text model trained from scratch adds none of its own. Comparing candidates within a group adds a consistent but small gain. Larger models and ensembles change accuracy by less than the noise of any single test run.

Limitations. The method sees the reasoning only through a fixed embedding and cannot check a solution step by step. Questions whose only correct answer has a single vote remain the bottleneck (64.9% solved). Ambiguous reference answers and the `####` parsing artefact set a floor on achievable accuracy that no selector can remove.